COMPUTER ENGINEERING
QUEZON CITY

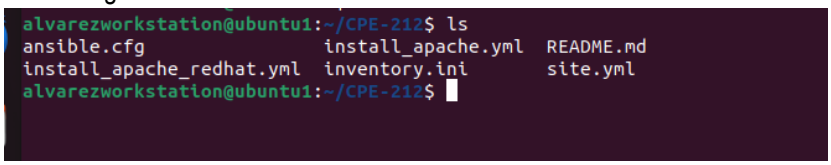
COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	03/09/2026	**DATE SUBMITTED:**	03/09/2026
NAME:	Alvarez, Erick Jefferson A	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 **Targeting Specific Nodes**

1. PROCEDURE OUTPUT

Task 1: Targeting Specific Nodes

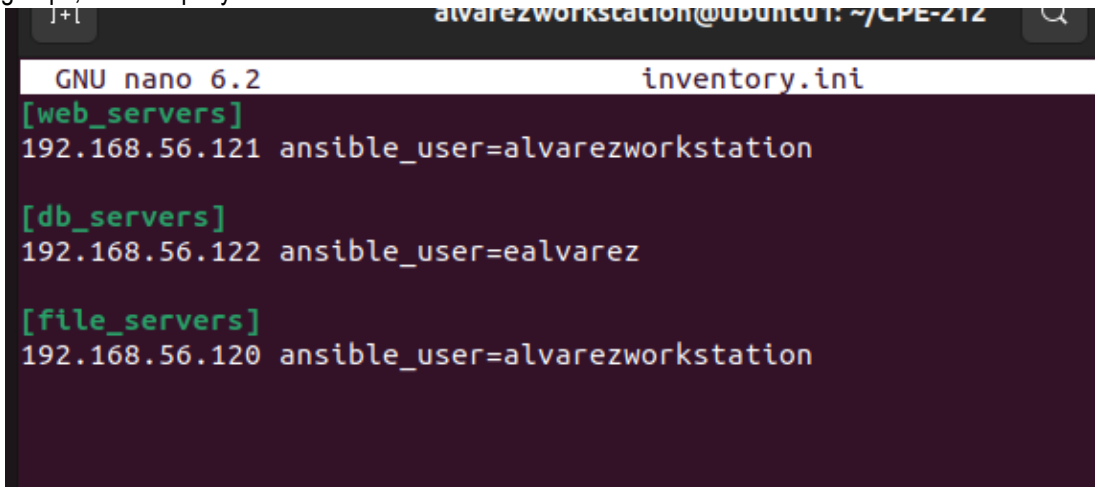
1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.



2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

Make sure to save the file and exit.

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.



3. Edit the site.yml by following the image below:

```
hosts: web_servers
become: true
tasks:
- name: install apache and php for Ubuntu servers
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: install apache and php for CentOS servers
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

The pre-tasks command tells the ansible to run it before any other thing. In the pre-tasks, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at web_servers. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

```
alvarezworkstation@ubuntu1: ~/CPE-212
  GNU nano 6.2                          site.yml
---
- hosts: all
  become: true
  pre_tasks:

  - name: install updates (CentOS)
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

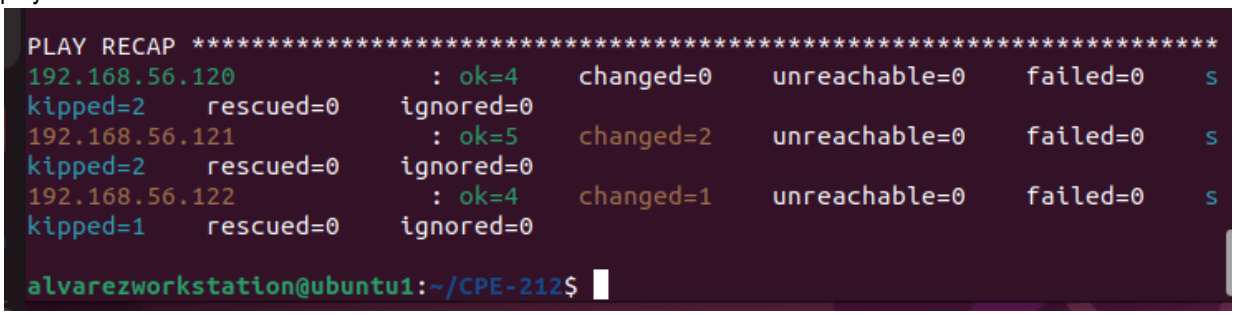
- hosts: web_servers
  become: true
  tasks:
```

Run the site.yml file and describe the result.



- The "site.yml" is functioning, but an error was encountered. After debugging, I realized that the error was only within my "inventory.ini." But clearly, the playbook is functioning.

4. Let's try to edit again the site.yml file. This time, we are going to add plays targeting the other servers. This time we target the db_servers by adding it on the current site.yml. Below is an example: (Note add this at the end of the playbooks from task 1.3.



Make sure to save the file and exit.

Run the site.yml file and describe the result.

- As can be seen from the figure above, after adding the db_servers group, the database server was handled by Ansible. The MariaDB was installed on the server based on its operating system.

5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: systemctl status mariadb. Do this on the CentOS server also.
Ubuntu:

```
*** System restart required ***
Last login: Thu Sep  3 09:05:07 2026 from 192.168.56.119
alvarezworkstation@server2:~$ systemctl status mariadb
Unit mariadb.service could not be found.
alvarezworkstation@server2:~$ systemctl status mariadb
Unit mariadb.service could not be found.
alvarezworkstation@server2:~$ exit
Logout
```

CentOS:

```
Last login: Thu Sep  3 09:03:34 2026 from 192.168.56.119
[ealvarez@alvarezworkstation ~]$ systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: >
     Active: active (running) since Thu 2026-09-03 08:58:42 PST; 12min ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 6187 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, >
    Process: 6209 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.serv>
    Process: 6310 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited>
   Main PID: 6295 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 9 (limit: 22981)
     Memory: 81.5M (peak: 100.6M)
```

Describe the output.

- This command reveals the current status of the MariaDB service. It is active (running) when the service is running.

6. Edit the site.yml again. This time we will append the code to configure installation on the file_servers group. We can add the following on our file.

```
PLAY RECAP *********************************************************************
192.168.56.120             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.121             : ok=5    changed=2    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.122             : ok=4    changed=1    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

Make sure to save the file and exit.

Run the site.yml file and describe the result.

- From the figure above, after adding the file_servers group, Ansible worked on the server under that group and installed the Samba package.

The testing of the file_servers is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we

can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the site.yml file. Add tags to the playbook. After the name, we can place the tags: name_of_tag. This is an arbitrary command, which means you can use any name for a tag.

```
PLAY RECAP *********************************************************************
192.168.56.120             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.121             : ok=5    changed=1    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.122             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

Make sure to save the file and exit.

Run the site.yml file and describe the result.

- The addition of tags enabled me to pick the parts of the playbook to be executed.

2. On the local machine, try to issue the following commands and describe each result:

2.1 ansible-playbook --list-tags site.yml

- This command outputs the tags present in the playbook.

```
alvarezworkstation@ubuntu1:~/CPE-212$ ansible-playbook --list-tags site.yml

playbook: site.yml

  play #1 (all): all    TAGS: []
      TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
      TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

  play #3 (db_servers): db_servers      TAGS: []
      TASK TAGS: [centos. db, db, mariadb, ubuntu]

  play #4 (file_servers): file_servers  TAGS: []
      TASK TAGS: [samba]
alvarezworkstation@ubuntu1:~/CPE-212$
```

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

- This command executes only those tasks that are tagged centOS.

```
PLAY RECAP *********************************************************************
192.168.56.120             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.121             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.122             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

2.3 ansible-playbook --tags db --ask-become-pass site.yml

- This command executes only those tasks that are tagged db.

```
PLAY RECAP *********************************************************************
192.168.56.120             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.121             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.122             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

- This command executes only those tasks that are tagged apache.

```
PLAY RECAP *********************************************************************
192.168.56.120             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.121             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.122             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

2.5 ansible-playbook --tags “apache,db” --ask-become-pass site.yml

- This command executes the tasks associated with both apache and db tags.

```
PLAY RECAP *********************************************************************
192.168.56.120             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.121             : ok=4    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.122             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

alvarezworkstation@ubuntu1:~/CPE-212$
```

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache, centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"

hosts: db_servers
```

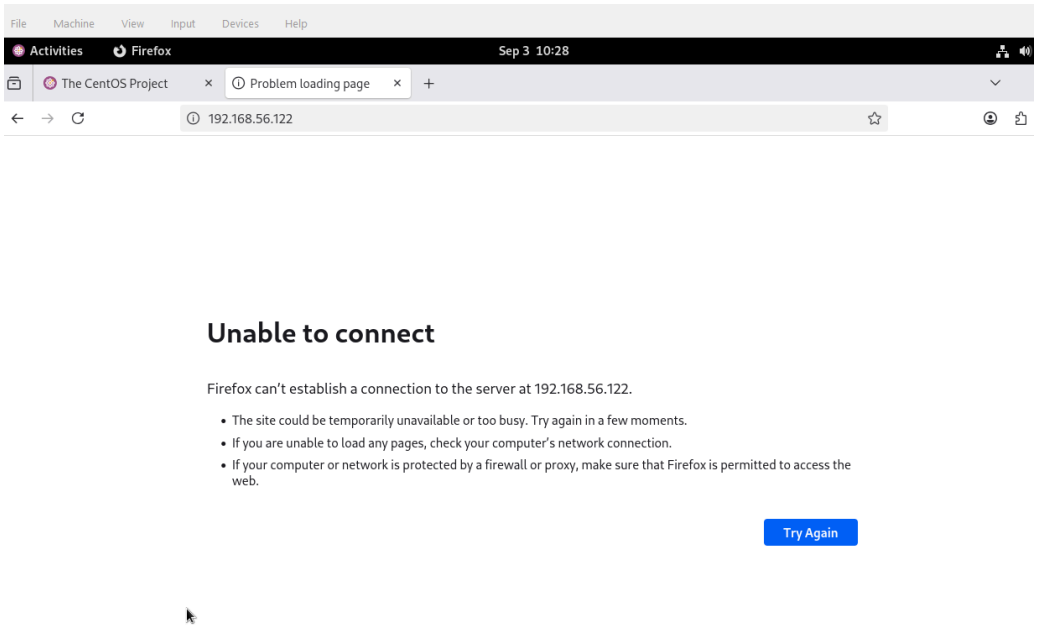
Figure 3.1.1
Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

```
- name: "Mariadb- Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true
```

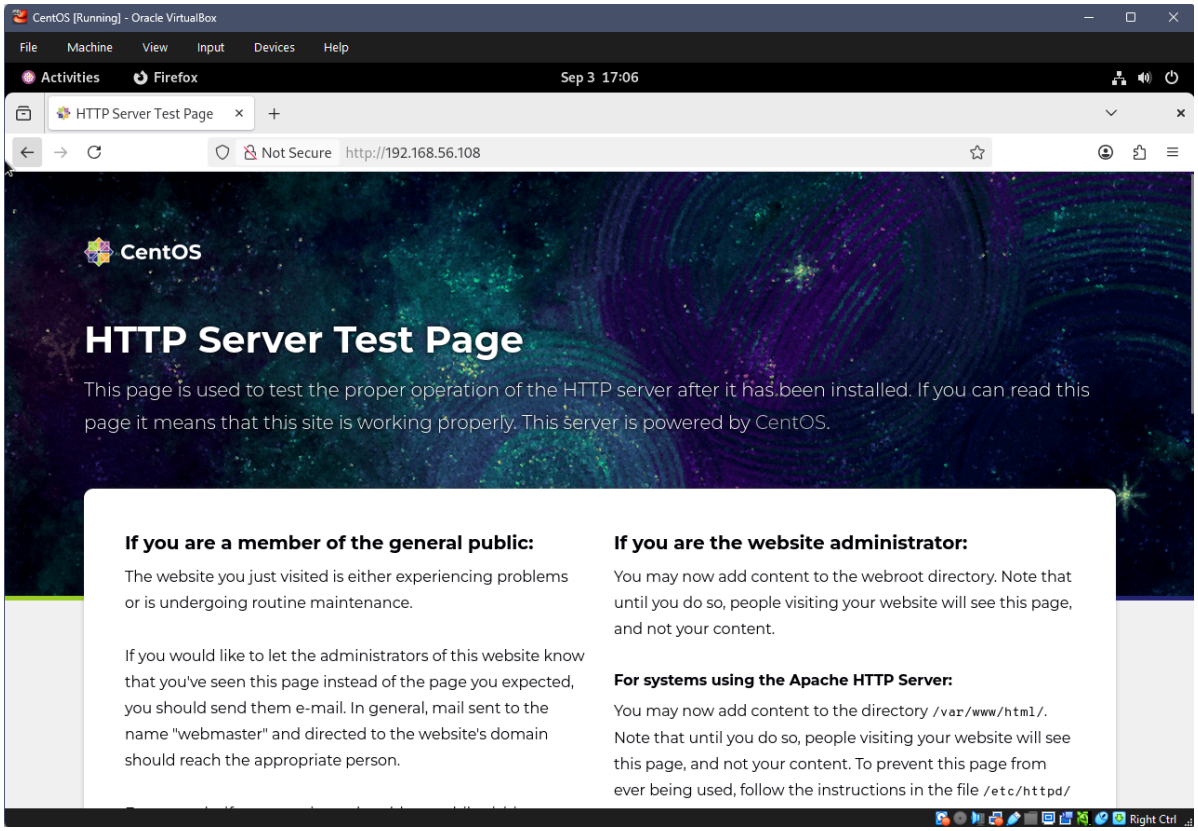
Figure 3.1.2
This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command sudo systemctl stop httpd. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.



3. Go to the local machine and this time, run the site.yml file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.

- After starting the playbook, Ansible will restart the httpd service. Now if we visit again the IP address, we can see below that the page or httpd is working again



To automatically enable the service every time we run the playbook, use the command enabled: true similar to Figure 7.1.2 and save the playbook.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

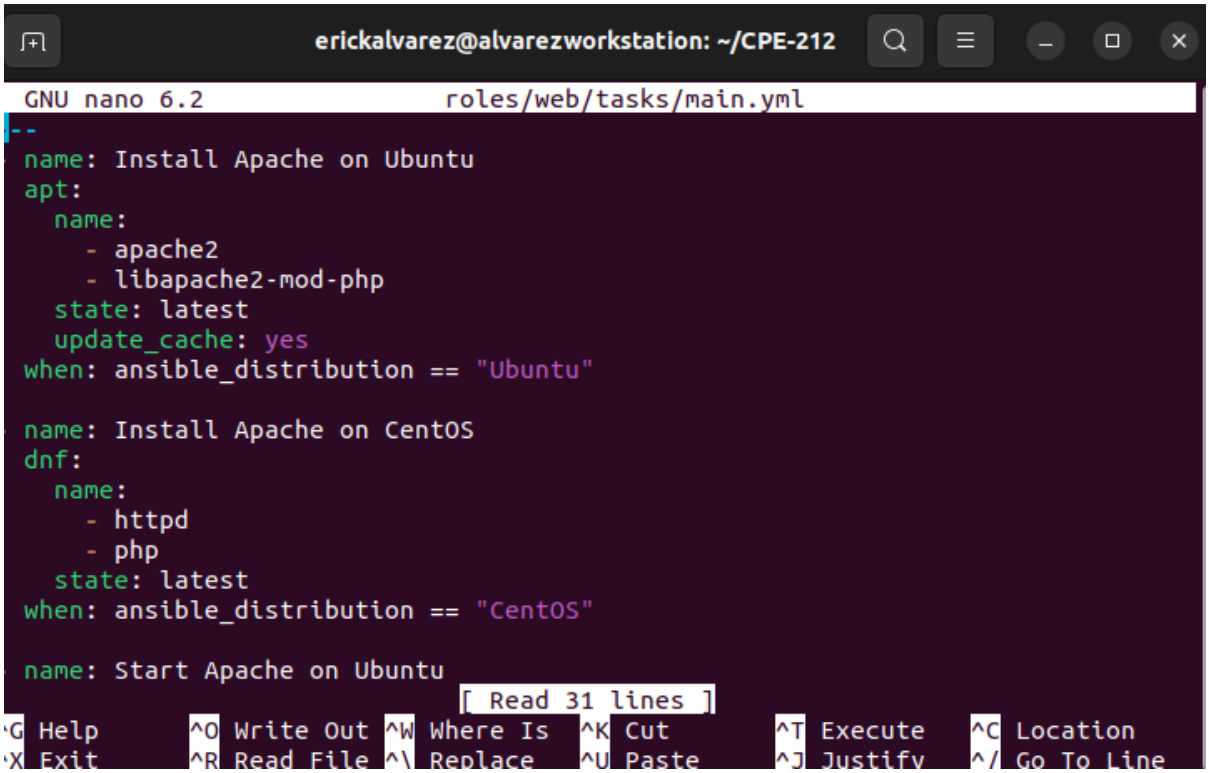
Supplementary Activity:

1. Creating a Role for Web Servers
2. Creating a Role for Web Servers
3. Creating a Role for Database Servers

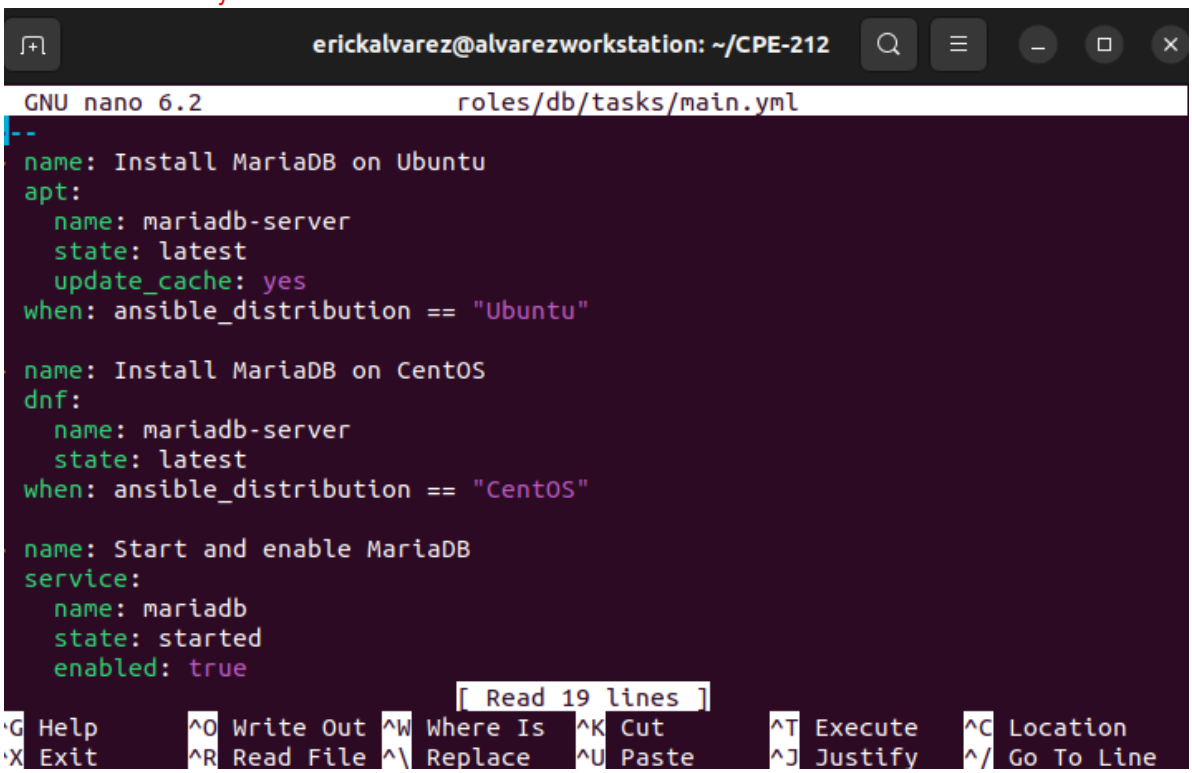
```
erickalvarez@alvarezworkstation:~/CPE-212$ ls roles
db  web
erickalvarez@alvarezworkstation:~/CPE-212$
```

4. Combining Multiple Roles in a Playbook

Web server playbook:

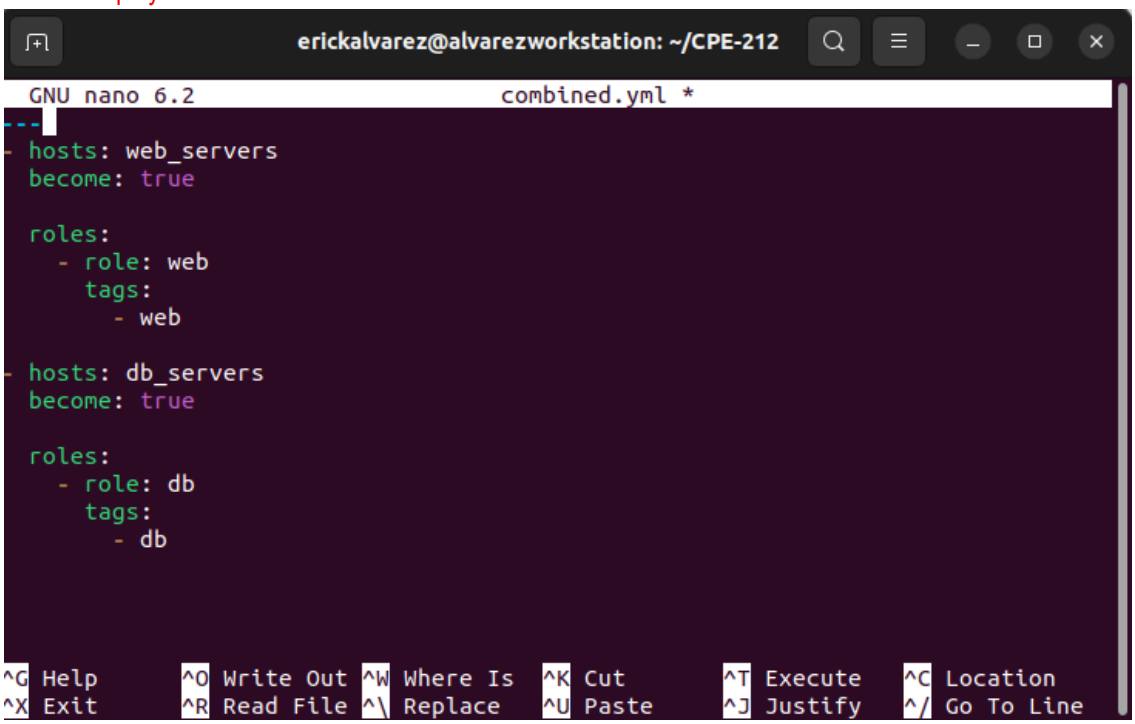


Database server Playbook:



To orchestrate multiple roles in a single playbook for different server groups.
Add tags to each role to allow selective execution, e.g., tags: web for webserver,
tags: db for database.

Combined playbook:



After running:



```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.107             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.108             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

Using –tags web:

```
TASK [web : Install Apache on Ubuntu] ******************
skipping: [192.168.56.108]
ok: [192.168.56.105]

TASK [web : Install Apache on CentOS] ******************
skipping: [192.168.56.105]
ok: [192.168.56.108]

TASK [web : Start Apache on Ubuntu] ********************
skipping: [192.168.56.108]
ok: [192.168.56.105]

TASK [web : Start httpd on CentOS] *********************
skipping: [192.168.56.105]
ok: [192.168.56.108]
```

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
192.168.56.107             : ok=1    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.56.108             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

Using –tags db

```
PLAY [db_servers] ********************************

TASK [Gathering Facts] ***************************
ok: [192.168.56.107]

TASK [db : Install MariaDB on Ubuntu] ************
ok: [192.168.56.107]

TASK [db : Install MariaDB on CentOS] ************
skipping: [192.168.56.107]

TASK [db : Start and enable MariaDB] *************
ok: [192.168.56.107]
```

```
PLAY RECAP *********************************************************************
192.168.56.105             : ok=1    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0
192.168.56.107             : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
192.168.56.108             : ok=1    changed=0    unreachable=0    failed=0    s
kipped=0    rescued=0    ignored=0

erickalvarez@alvarezworkstation:~/CPE-212$
```

All tags:

```
erickalvarez@alvarezworkstation:~/CPE-212$ ansible-playbook combined.yml --list-
tags

playbook: combined.yml

  play #1 (web_servers): web_servers    TAGS: []
      TASK TAGS: [web]

  play #2 (db_servers): db_servers      TAGS: []
      TASK TAGS: [db]
erickalvarez@alvarezworkstation:~/CPE-212$
```

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?
   - The grouping of the servers enables their management according to the function that they perform. Group names such as web_servers, db_servers and file_servers can be created. Rather than selecting servers one by one, Ansible runs the necessary tasks for the particular group.

2. What is the importance of tags in playbooks?
   - The tagging gives me the option to select the required tasks to be run. Tagging will be useful when there is a need to change something specific on the server rather than running the whole playbook.

3. Why do think some services need to be managed automatically in playbooks?
   - Some of the services are not started automatically after their installation. This situation is very common in CentOS. During execution of the playbook, it is possible to manage the services without starting them each time manually.

Conclusion:
In conclusion, the activity made me understand how the management of servers becomes simpler using Ansible. Some of the things I was able to learn included how to create server groups, use tags, manage services and create roles for different functions. I also faced the challenge of wrong inventory with my inventory.ini file, and from this, I have been able to understand the importance of having the right inventory.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.